

DBT TRAINING

● Working Effectively · Module 11 · 45 min

Selectors, Tags, and Running Subsets

Stop rebuilding everything. Run exactly what you need.

The Problem: `dbt run` Takes 8 Minutes

Model	Layer	Run time	Changed today?
<code>stg_hubspot__contacts</code>	Staging	4 sec	✗ No
<code>stg_hubspot__pipeline_stages</code>	Staging	3 sec	✗ No
<code>stg_hubspot__deals</code>	Staging	4 sec	✗ No
<code>stg_products</code>	Staging	2 sec	✗ No
<code>stg_prescriptions</code>	Staging	3 sec	✗ No
<code>dim_contact</code>	Silver	45 sec	✗ No
<code>dim_pipeline_stage</code>	Silver	12 sec	✗ No

The Selection Syntax

`--select` — choose what to run

```
# By model name
dbt run --select fct_deal

# By directory / layer
dbt run --select silver.*
dbt run --select staging.*

# By tag
dbt run --select tag:daily

# By source
dbt run --select source:hubspot

# Graph operators
dbt run --select +fct_prescription
dbt run --select mrt_deals_funnel+
dbt run --select +fct_prescription+
```

`--exclude` — subtract from the selection

```
# All Silver, except fct_prescription
dbt run \
  --select silver.* \
  --exclude fct_prescription

# Daily tag, skip Gold
dbt build \
  --select tag:daily \
  --exclude gold.*

# Combine freely
dbt build \
  --select +fct_deal \
  --exclude staging.*
```

`--exclude` accepts the same syntax as `--select`. dbt selects first, then removes excluded nodes.

Selection Methods at a Glance

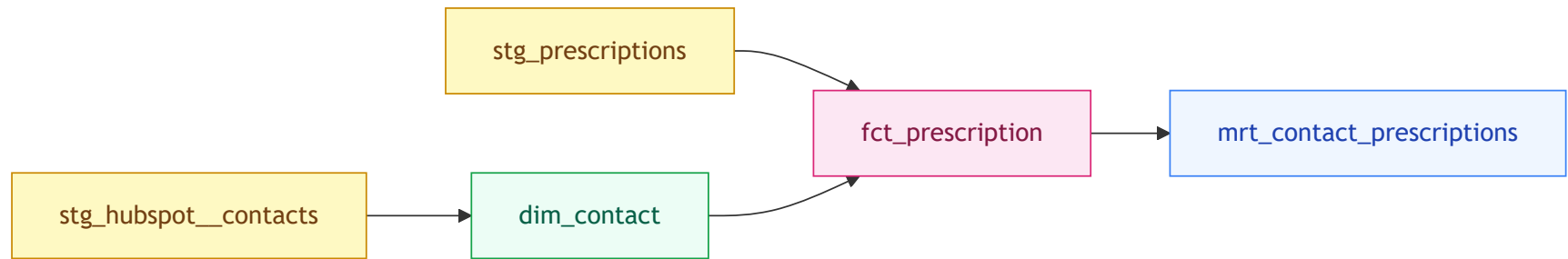
Method	Syntax	Example	Selects
By name	<code>model_name</code>	<code>fct_deal</code>	That model only

Method	Syntax	Example	Selects
By directory	<code>folder.*</code>	<code>silver.*</code>	All models in that folder

Method	Syntax	Example	Selects
By tag	<code>tag:name</code>	<code>tag:daily</code>	All models with that tag

Method	Syntax	Example	Selects
By source	<code>source:name</code>	<code>source:hubspot</code>	Models that source from that source

Graph Operators — Walking the DAG



+fct_prescription

fct_prescription + all upstream: stg_contacts, stg_prescriptions, dim_contact

fct_prescription+

fct_prescription + all downstream: mrt_contact_prescriptions

+fct_prescription+

fct_prescription + all upstream + all downstream: all 5 models

Memory aid: the `+` is always on the side where the graph extends. Prefix `+model` → extends toward parents (upstream). Suffix `model+` → extends toward children (downstream).

Depth Limiting: `1+model` and `model+2`

Without a depth number, `+model` walks the entire upstream graph — potentially dozens of models. In a large project, `+fct_prescription` might match 20 upstream models. If you know only the direct staging parent is stale, use `1+fct_prescription` to limit to one level up — faster and cheaper.

Add a number to limit how many levels you traverse:

```
# No limit — full upstream chain
dbt run --select +fct_prescription
# Selects: stg_contacts, stg_prescriptions,
#         dim_contact, fct_prescription

# 1 level upstream only
dbt run --select 1+fct_prescription
# Selects: dim_contact, stg_prescriptions,
#         fct_prescription
# (stg_contacts is 2 levels up — excluded)

# 2 levels downstream from dim_contact
dbt run --select dim_contact+2
# Selects: dim_contact, fct_prescription,
#         mrt_contact_prescriptions
```

When to use depth limits

- **1+model** — rebuild a model with only its direct parents. Use when you know the grandparent layer is already fresh.
- **model+N** — limit blast radius when testing a change. See 2 levels downstream without running the full Gold layer.

Depth limits are less common than plain operators. Know they exist — reach for them when an unbounded `+` selects far more than you intended.

Tags — Categorizing Models

In `dbt_project.yml` — folder-level defaults

```
models:
  analytics:
    staging:
      +tags: ['staging']
  silver:
    +tags: ['silver']
    facts:
      +tags: ['daily']
  gold:
    +tags: ['gold', 'weekly']
```

The `+` prefix cascades the tag down to all models in the folder. A model in `silver/facts/` inherits both `silver` and `daily`.

In `schema.yml` — model-level tags

```
models:
  - name: fct_deal
    config:
      tags: ['daily', 'finance']
  - name: mrt_deals_funnel
    config:
      tags: ['weekly', 'finance']
```

Or inline in the model:

```
{{ config(tags=['daily', 'finance']) }}
```

Tags stack. `fct_deal` has `daily` from the folder rule and `finance` from `schema.yml`. Both selectors work.

Scheduling use case: Once models are tagged, your orchestrator (Airflow, GitHub Actions cron) can select by tag instead of model names. When you add a new `fct_*` model and tag it `daily`, the

dbt build vs dbt run vs dbt test

Command	Runs models?	Runs tests?	Runs seeds?	Runs snapshots?	When to use
dbt run	✓	✗	✗	✗	Fast dev iteration — see transformed data quickly
dbt test	✗	✓	✗	✗	Validate after dbt run, or retest after a data fix
dbt build	✓	✓	✓	✓	CI pipelines, production runs — our standard

Why dbt build is the default: Tests run immediately after each model in dependency order. If `fct_prescription` produces a NULL in a `not_null`-tested column, `dbt build` fails that node before `mrt_contact_prescriptions` runs on bad data. `dbt run` would continue and produce a corrupted Gold mart.

result:error — Recovering From Partial Failures

A `dbt build` failed on 3 models. 10 others succeeded. You don't want to rerun the clean ones. Why not just re-run `dbt build --select silver.*`? Because 7 models already succeeded. Running them again wastes 3 minutes. `result:error` skips the clean models and runs only the failed ones.

Step 1 — The original run (partial failure)

```
dbt build --select silver.*

# Output:
# 7 models built. 0 errors.      ← OK
# 0 of 4 tests passed.         ← Tests failed
# 3 models error.              ← These need fixing
```

What lives in `./target`

- `manifest.json` — the compiled DAG for this run
- `run_results.json` — status of every node (pass / error / skip)
- `compiled/` — the SQL dbt generated for each model

Step 2 — Fix the root cause, then:

```
dbt run --select result:error \
      --state ./target
```

Important: `./target` is overwritten on every run. If you run anything between the failure and the retry, the state is lost. Run the retry immediately after fixing the issue — or copy `run_results.json` before rerunning.

dbt reads `target/run_results.json` from the previous run, finds the errored nodes, and selects

Exercise — Write the Selection Command (15 min)

Five scenarios. Write the exact `dbt` command for each. Use the exercise project model names. Work solo — we'll debrief together at minute 40.

Scenario 1

"Rebuild only the Silver fact models and their tests."

Scenario 2

"Run `fct_prescription` and everything it depends on."

Scenario 3

"Run `mrt_deals_funnel` and all models downstream of it."

Scenario 4

"Run all daily-tagged models but skip Gold marts."

Scenario 5

"After a partial failure, rerun only the models that errored."

Key Takeaways



Select precisely, not broadly

`dbt run --select +fct_deal` rebuilds exactly what changed and its inputs. `dbt run` rebuilds everything. One is a scalpel; the other is a sledgehammer.



Tags decouple model names from your scheduler

Tag your models by cadence (`daily` , `weekly`) and domain (`finance`). Schedulers select by tag — when you add a model, the schedule picks it up automatically without touching pipeline config.



`dbt build` is the default for production

It runs models, tests, seeds, and snapshots in dependency order. Tests run immediately after each model — bad data doesn't propagate downstream. Use `dbt run` only for fast local iteration.

MODULE 11 COMPLETE

Next: Module 12

CI/CD with dbt — Slim Runs, PR Checks, and State Management

Prep Q1: What is the difference between dbt run and dbt build in a CI pipeline context?

Prep Q2: If a PR changes dim_contact, which downstream models might be affected — and how would you find out?

Prep Q3: What is a "slim CI" run, and why would you want it instead of dbt build --select silver.*?

Prep Q4: What does --defer do, and why would a CI job use it instead of rebuilding every model from scratch?